

GOCHANO — COMPLETE CLEAN MINIMALIST UI/UX REBUILD, FRONTEND RESTRUCTURE, FEATURE VERIFICATION & FINAL PROJECT CLEANUP

You are acting as a:

- Senior Flutter Architect
- Senior Product Designer
- Senior UX/UI Engineer
- Backend Integration Auditor
- QA Engineer
- Production Readiness Engineer

You are working on an EXISTING application called:

GOCHANO

Gochano is a student-centered productivity, academic-management, life-management, community, AI-assistance, medicine, expense, and Bangladesh commute application.

This is NOT a new project.

The application is already substantially implemented.

The backend, databases, authentication, AI services, private file storage, CommuteBD datasets, and most business logic are already connected.

Your task is to:

1. Completely redesign the application's UI/UX.
2. Completely reorganize the frontend information architecture.
3. Make the application significantly easier to understand and use.
4. Make it professional enough to look like a real production application.
5. Keep it student-friendly without making it childish.
6. Use Clean Minimalism as the primary UI style.
7. Use professionally designed STATIC illustrated/cartoon-inspired icons and illustrations where appropriate.
8. DO NOT introduce decorative animations.
9. Verify every existing feature from frontend to backend.
10. Verify AI Assistance thoroughly.
11. Verify Study Groups, chat, and resource sharing thoroughly.
12. Verify CommuteBD routing, Dijkstra usage, fare prediction and existing ML integration.

13. Verify Prescription OCR and determine whether a custom ML model actually exists and is being used.
 14. Repair disconnected/broken integrations where possible without changing the backend architecture or public API contracts.
 15. Preserve working existing features.
 16. Remove duplicate/outdated README files.
 17. Leave exactly ONE final root README.md.
 18. Make the final README.md the complete project report, developer manual, owner setup guide, architecture documentation, feature documentation, troubleshooting guide, deployment guide, and verified project-status report.
-

SECTION 1 — CRITICAL RULE: THIS IS NOT A REBUILD FROM SCRATCH

THIS PROJECT ALREADY EXISTS.

DO NOT:

- create a second Flutter application;
- start another backend;
- replace FastAPI;
- replace Firebase;
- replace Firestore;
- replace Neon PostgreSQL;
- replace PostGIS;
- replace Backblaze B2;
- replace Gemini;
- replace Render;
- replace Firebase Authentication;
- redesign the backend architecture unnecessarily;
- arbitrarily change API endpoints;
- arbitrarily rename API fields;
- break Firestore document structures;
- break existing PostgreSQL structures;
- remove existing working features;
- replace working services simply because another implementation appears cleaner;
- introduce Supabase;
- introduce Firebase Storage as the primary file provider;
- expose backend secrets inside Flutter;

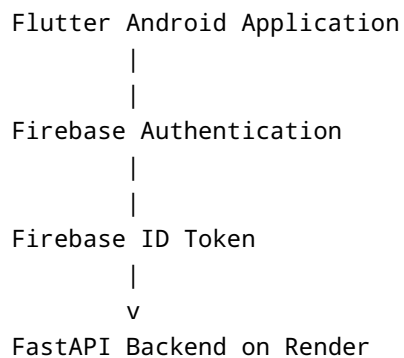
- This is an:

+ FULL FRONTEND REDESIGN

+ INTEGRATION VERIFICATION

+ FINAL PROJECT CLEANUP.

Treat the existing codebase as the primary source of truth and verify actual implementation before making claims.



```
|
+----- Firebase Admin
|
+----- Cloud Firestore
|
+----- FCM
|
+----- Neon PostgreSQL
|
+----- PostGIS
|
+----- Backblaze B2
|
+----- Google Gemini
|
+----- OSRM / OpenStreetMap / Nominatim
|
+----- Existing routing / ML components
```

Core technologies:

Frontend

- Flutter
- Dart
- Material 3 foundations
- Custom Clean Minimalist design system

Backend

- Python
- FastAPI

Authentication

- Firebase Authentication

Application / Realtime Data

- Cloud Firestore

Relational / Transport / Geographic Data

- Neon PostgreSQL
- PostGIS

Private File Storage

- Backblaze B2

Artificial Intelligence

- Google Gemini API

CommuteBD

- Bangladesh transport datasets
- PostgreSQL/PostGIS
- OpenStreetMap
- OSRM where integrated
- Nominatim where integrated
- Dijkstra or existing graph-routing implementation
- Existing ML fare model if actually available and connected

OCR

Inspect actual implementation.

Potential components may include:

- Tesseract
- pdf2image
- Pillow
- image preprocessing
- PDF extraction
- rules / regex
- Gemini-assisted structured extraction
- custom ML models

DO NOT ASSUME all of these exist.

Verify them.

Deployment

- Render

Platform

- Android

Languages

- English
- Bangla

SECTION 3 — WHAT MAY CHANGE

You MAY significantly change:

- overall UI;
- page composition;
- screen layout;
- frontend navigation;
- frontend information hierarchy;
- bottom navigation;
- component hierarchy;
- card design;
- icons;
- illustrations;
- typography;
- colors;
- spacing;
- forms;
- dialogs;
- bottom sheets;
- empty states;
- loading states;
- error states;
- frontend folder organization;
- reusable UI architecture;
- design tokens;
- presentation-layer state handling;
- accessibility;
- responsive layout;
- frontend interaction flow.

The application is EXPECTED to look significantly different when finished.

SECTION 4 — WHAT MUST NOT CHANGE

Do not break or unnecessarily replace:

- existing FastAPI API contracts;
- Firebase Authentication;
- Firebase ID-token flow;
- Firestore ownership model;
- Firestore production data;
- Neon database;
- PostGIS structures;
- CommuteBD datasets;
- Backblaze B2 file-storage ownership;
- Gemini service architecture;
- authentication headers;
- security checks;
- functioning backend services;
- existing business rules;
- user accounts;
- user files;
- study materials;
- study groups;
- chat records;
- expenses;
- medicines;
- transport data;
- preferences.

The target is:

NEW FRONTEND EXPERIENCE

+ NEW FRONTEND STRUCTURE

+ BETTER USABILITY

+ SAME CONNECTED PRODUCT.

SECTION 5 — CONTROLLED BACKEND REPAIR RULE

The backend architecture must remain stable.

However, feature verification may reveal an existing feature that is incorrectly connected.

Examples:

- an ML model exists but is never loaded;
- Dijkstra exists but the route endpoint bypasses it;
- OCR code exists but Flutter calls another endpoint;
- B2 storage exists but Firebase Storage functions override it;
- group resource API supports PDFs but Flutter file picker rejects PDFs;
- group chat exists but messages are not rendered;
- Gemini material context exists but Flutter sends the wrong material ID;
- Firebase token is missing from a specific request;
- a valid backend endpoint is called with the wrong request body;
- fare-prediction model exists but a fallback always replaces its result;
- database query is broken;
- an endpoint uses an old path.

In these cases you MAY make the minimum necessary internal repair.

Prefer:

```
Existing endpoint
+
Existing request schema
+
Existing response schema
+
Correct internal implementation
```

Do not redesign the entire backend.

SECTION 6 — COMPLETE REPOSITORY AUDIT FIRST

BEFORE changing the UI, inspect the complete repository.

Flutter Audit

Inspect:

- lib/
- screens
- pages
- widgets
- models
- providers
- repositories
- controllers
- services
- network classes
- routing
- state management
- authentication
- theme
- localization
- assets
- illustrations
- icons
- fonts
- shared widgets
- Firebase initialization
- notifications
- local storage
- caching
- file picker
- image picker
- PDF handling
- permission handling
- deep links if implemented
- existing loading/error states

Backend Audit

Inspect:

- main.py
- routers
- schemas
- models
- services
- dependencies
- authentication
- Firebase Admin
- Firestore
- Backblaze B2
- Gemini
- OCR
- PDF extraction
- Study services
- Group services
- chat
- resources
- Expense
- Medicine
- CommuteBD
- PostgreSQL
- PostGIS
- routing algorithms
- Dijkstra
- OSRM
- Nominatim
- fare calculation
- fare prediction
- ML inference
- environment configuration
- tests

SECTION 7 — FIND EVERY ML MODEL

Search for:

```
.pkl  
.pickle  
.joblib  
.onnx  
.pt  
.pth  
.tflite  
.h5  
.keras
```

Also search code for:

- sklearn
- TensorFlow
- PyTorch
- XGBoost
- LightGBM
- CatBoost
- model.predict
- inference
- load_model
- joblib.load
- pickle.load
- shortest_path
- dijkstra
- graph
- networkx

For every model discovered determine:

1. What is it?
2. Where is it stored?
3. How is it loaded?
4. What inputs does it require?
5. What output does it produce?
6. Which backend service calls it?
7. Which endpoint calls that service?
8. Which Flutter screen calls that endpoint?
9. Is inference actually happening at runtime?
10. Is a fallback overriding the prediction?
11. Is the model trained from a real dataset?
12. Is the result used by the user-facing feature?

DO NOT claim "ML-powered" simply because model-related code exists.

SECTION 8 — TRACE FEATURES END TO END

Do not assume that a feature works because a button or function exists.

Trace:

```
Flutter UI
↓
Flutter state/controller
↓
service/repository
↓
HTTP request
↓
FastAPI route
↓
backend service
↓
database / storage / AI / ML
↓
response
↓
Flutter parser
↓
Flutter state
↓
user-visible result
```

Do this for all critical features.

SECTION 9 — PRIMARY DESIGN LANGUAGE

The new Gochano interface should use:

CLEAN MINIMALISM

The product should feel:

- professional;
- calm;
- modern;
- clean;
- focused;
- organized;
- trustworthy;
- spacious;
- student-friendly;
- simple to navigate;
- easy to understand;
- visually pleasant.

It should NOT look:

- childish;
- like a game;
- excessively colorful;
- excessively cartoonish;
- like an admin dashboard;
- like cryptocurrency software;
- overly futuristic;
- excessively decorative;
- filled with unnecessary gradients;
- filled with huge shadows;
- cluttered.

SECTION 10 — STATIC CARTOON-INSPIRED VISUAL LANGUAGE

Where visual personality improves usability, use:

- static vector illustrations;
- static illustrated icons;
- soft cartoon-inspired icons;
- simple editorial-style illustrations;
- clean 2D artwork;
- simplified student-themed visual metaphors.

IMPORTANT:

These graphics must be **STATIC**.

DO NOT make them move.

DO NOT use decorative animated icons.

DO NOT use animated illustrations.

DO NOT use Rive.

DO NOT use Lottie.

DO NOT add custom motion effects.

The goal is:

```
Clean Minimalism
+
Soft static illustration
+
Professional student personality
```

NOT:

Cartoon application

Illustrations should support understanding rather than dominate the interface.

SECTION 11 — ABSOLUTELY NO DECORATIVE ANIMATION SYSTEM

Do not introduce:

- animated subject icons;
- animated note icons;
- animated buttons;
- animated cards;
- animated counters;
- animated navigation;
- custom page transitions;
- Hero animations for decoration;
- bouncing icons;
- spinning illustrations;
- pulsing illustrations;
- floating illustrations;
- animated route drawing;
- animated checkmarks;
- decorative fade entrances;
- decorative slide entrances;
- decorative scale transitions;
- Rive animations;
- Lottie animations;
- particle effects;
- animated backgrounds;
- shimmer purely for visual decoration.

Do not create a motion-design system.

Prefer instant, clear state changes.

Standard Android/Flutter framework behavior that is unavoidable for core interaction is acceptable, but do not intentionally add animation as a design feature.

SECTION 12 — FUNCTIONAL LOADING FEEDBACK

Loading must still be understandable.

Prefer:

- static loading placeholder;
- text such as "Loading materials...";
- determinate file-upload percentage where available;
- simple static progress indicator where possible;
- disabled action while processing;
- clear AI processing message;
- clear route-calculation message;
- clear OCR-processing message.

Do not add visually distracting animated loaders purely for aesthetics.

SECTION 13 — MATERIAL 3 FOUNDATION

Use Material 3 primarily for:

- interaction patterns;
- accessibility;
- forms;
- fields;
- dialogs;
- bottom sheets;
- navigation foundations;
- buttons;

- menus;
- controls.

Do not make the application look like untouched default Material widgets.

Build a custom Gochano design system above Material 3.

SECTION 14 — BENTO USAGE

Bento-style grouping may be used selectively for:

- Home overview;
- Study overview;
- Expense summaries;
- Profile statistics.

Do NOT make every screen a Bento grid.

Content-heavy screens such as:

- material lists;
- group chat;
- PDF reading;
- expense history;
- medicine history;

should use normal readable layouts.

SECTION 15 — CENTRAL DESIGN SYSTEM

Create one reusable frontend design system.

Centralize:

Colors

- primary;
- secondary;
- accent;
- background;
- surface;
- surface variant;
- elevated surface;
- border;
- divider;
- text primary;
- text secondary;
- disabled;
- success;
- warning;
- error;
- information.

Module accent colors may exist but must remain restrained.

Typography

Create clear tokens for:

- display;
- page title;
- section heading;
- card heading;
- body;
- secondary body;
- button;
- label;
- caption;
- numeric/statistics.

Spacing

Use a consistent scale such as:

4
8
12
16

20
24
32
40

Radius

Use consistent:

- small;
- medium;
- large;
- extra-large.

Avoid random corner radii.

Shadows

Keep subtle.

Prefer:

- spacing;
- background separation;
- borders;

over heavy shadows.



SECTION 16 — COLOR STRATEGY



Use one recognizable Gochano brand identity.

Base UI:

- calm neutral surfaces;
- high readability;
- restrained accents.

Possible conceptual feature accents:

Study: blue / indigo

AI: soft violet

Expense: green

Medicine: teal

Commute: blue / cyan

Community: warm restrained accent

Do not make every feature card a different saturated color.

SECTION 17 — LIGHT MODE

Light mode should use:

- soft neutral background;
- white/slightly tinted surfaces;
- clear dark typography;
- subtle borders;
- restrained accent colors.

Avoid excessive pure-white empty space without hierarchy.

SECTION 18 — DARK MODE

Redesign dark mode properly.

Do not simply convert:

white → black.

Use:

- dark neutral background;
- slightly lighter surfaces;
- restrained borders;
- comfortable text contrast;
- muted accent colors.

Verify:

- every screen;
- card;
- dialog;
- bottom sheet;
- field;
- PDF controls;
- navigation;
- illustrations;
- icons;
- loading states;
- error states;
- empty states.

Static illustrations should remain readable in both light and dark themes.

If necessary provide theme-compatible illustration variants.

SECTION 19 — STATIC SUBJECT ICON SYSTEM

The Study system should have stronger visual identity.

Create a reusable component such as:

SubjectIllustration

or:

SubjectIcon

It must be STATIC.

Use soft illustrated/cartoon-inspired icons.

Example mapping:

Programming

Static illustration:

Laptop / code brackets

Artificial Intelligence

Small brain/neural-node illustration

Database

Database cylinder illustration

Networking

Connected computer/node illustration

Mathematics

Calculator / mathematical symbols

Physics

Atom illustration

Chemistry

Flask illustration

Biology

Cell / leaf / microscope

Software Engineering

Layered architecture / code blocks

English

Book / quotation / alphabet

Business

Chart / briefcase

Generic Subject

Notebook / books

These should look:

- modern;
- soft;
- friendly;
- illustrated;
- clean;
- professional.

They must NOT look like children's preschool stickers.

SECTION 20 — SUBJECT ICON FALLBACK

Allow custom subject names.

Use keyword matching only for selecting an appropriate visual.

Examples:

Artificial Intelligence
AI
Machine Learning

may use a similar AI illustration.

If no match exists:

use a generic study/book illustration.

Do not allow missing images or broken icon states.

SECTION 21 — STATIC RESOURCE ICONS

Use recognizable static illustrated icons for resources:

PDF: document/PDF illustration

Image: photo/image illustration

Text Note: notebook/pencil illustration

DOC/DOCX: document illustration

Presentation: slides illustration where supported

Other: generic file illustration

Keep the visual system consistent.

SECTION 22 — STATIC ILLUSTRATIONS FOR EMPTY STATES

Use small clean illustrations in appropriate empty states.

Examples:

No Subjects

Static student/books illustration.

Text:

No subjects yet
Create your first subject to organize your study materials.

No Materials

Static folder/notebook illustration.

No Tasks

Static checklist illustration.

No Group Resources

Static shared-folder illustration.

No Expenses

Static wallet/receipt illustration.

No Medicines

Static medicine-box illustration.

No Commute Result

Static map/location illustration.

Illustrations should not occupy excessive screen space.

SECTION 23 — RESPONSIVE UI

Primary target:

Android mobile.

Support:

- small phones;
- common phones;
- larger phones;
- system font scaling;
- keyboard;
- safe areas.

Prevent:

- RenderFlex overflow;
- clipped Bangla text;
- clipped buttons;
- keyboard-covered fields;
- bottom-navigation overlap;
- offscreen dialogs;
- broken grid layouts.

SECTION 24 — ACCESSIBILITY

Use:

- large touch targets;
- accessible contrast;
- meaningful labels;
- semantic widgets;
- readable typography;
- clear enabled/disabled states;
- icon labels where meaning is not obvious.

Do not use color alone to represent status.

SECTION 25 — FRONTEND ARCHITECTURE

Reorganize the PRESENTATION architecture using a practical feature-first structure.

Do not force a theoretical architecture that requires rewriting working services.

Recommended direction:

```
lib/  
|  
├─ app/  
|   ├─ app.dart  
|   ├─ bootstrap/  
|   └─ router/  
|  
├─ core/  
|   └─ design_system/
```

```
|
|
|   |— colors/
|   |— typography/
|   |— spacing/
|   |— radius/
|   |— theme/
|   |— icons/
|   |— illustrations/
|
|   |— localization/
|   |— navigation/
|   |— network/
|   |— constants/
|   |— utils/
|
|— shared/
|   |— widgets/
|   |— dialogs/
|   |— sheets/
|   |— states/
|
|— features/
|   |— home/
|   |   |— presentation/
|   |
|   |— study/
|   |   |— presentation/
|   |   |   |— workspace/
|   |   |   |— semesters/
|   |   |   |— subjects/
|   |   |   |— materials/
|   |   |   |— reader/
|   |   |   |— tasks/
|   |   |   |— planner/
|   |   |   |— focus/
|   |   |   |— groups/
|   |   |   |— ai/
|   |
|   |— life/
|   |   |— presentation/
|   |   |   |— expense/
|   |   |   |— medicine/
|   |   |   |— commute/
|   |
|   |— community/
|   |   |— presentation/
```

```
|
└─ profile/
    └─ presentation/
```

This is guidance, not permission to break existing architecture.

Preserve functioning:

- services;
- providers;
- models;
- controllers;
- repositories;
- state-management implementation.

Move/refactor only where safe.

SECTION 26 — MAIN INFORMATION ARCHITECTURE

Use five primary destinations:

HOME
STUDY
LIFE
COMMUNITY
PROFILE

Bottom navigation:

Home | Study | Life | Community | Profile

Do not create 8–10 permanent navigation items.

=====

SECTION 27 — HOME DASHBOARD

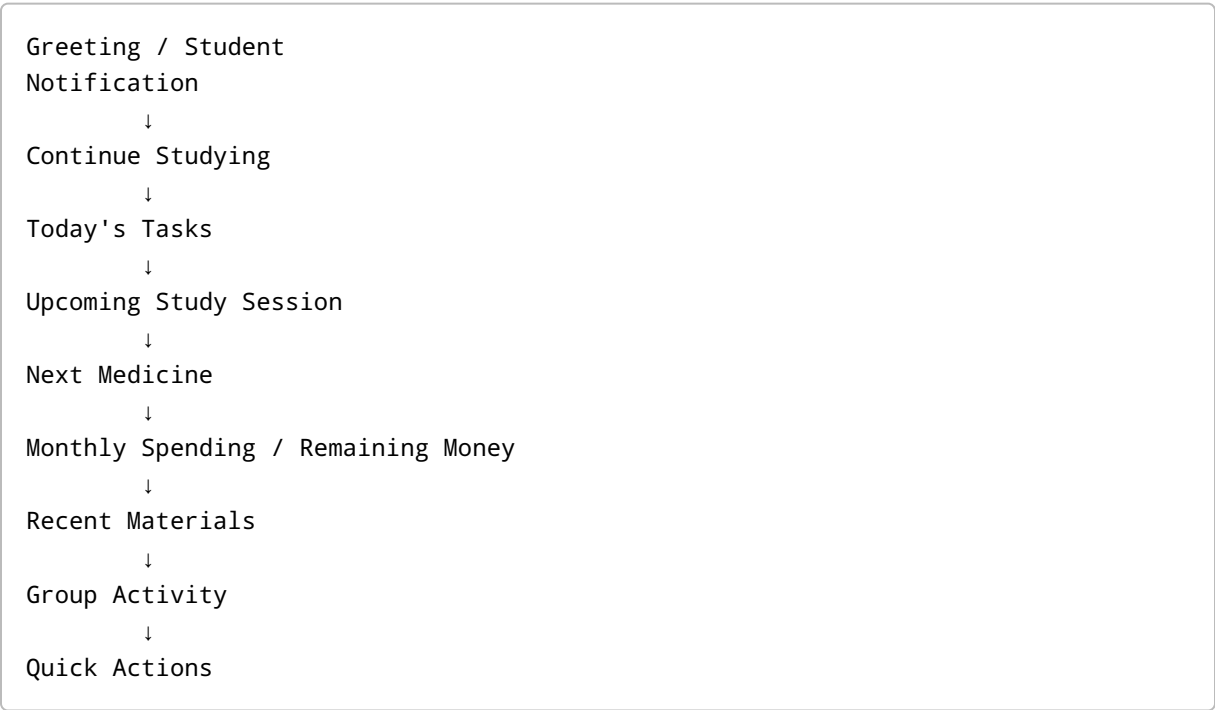
=====

Home must answer:

"What matters to me right now?"

Do NOT show every feature equally.

Recommended structure:



Quick Actions:

- Ask AI
- Add Expense
- Add Task
- Scan Prescription

- Start Commute

Avoid a huge 15-icon feature grid.

SECTION 28 — HOME STATIC VISUALS

Use a restrained static illustration where appropriate.

Possible Home visual:

A small student desk / books / laptop illustration.

Do not place giant hero art taking half the screen.

The illustration should support brand personality without reducing usability.

SECTION 29 — STUDY HUB

Study is the academic center of Gochano.

Organize:

```
Study
|
├─ Workspace
├─ Tasks
├─ Planner
├─ Focus
```



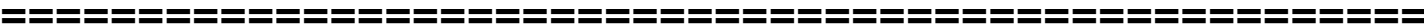
Workspace:



Surface Recent Materials to reduce navigation depth.



SECTION 30 — SEMESTER UI



Semester cards should show:

- semester name;
- number of subjects;
- recent activity where useful;
- simple static academic icon/illustration.

Primary action:



Secondary actions should be placed in an overflow/context menu.



SECTION 31 — SUBJECT UI

Each subject should have:

- name;
- static subject illustration;
- resource count;
- recent item;
- optional academic details;
- open action.

Subject screen should show:

- subject identity;
- materials;
- search;
- sort/filter;
- Add Material;
- Ask AI;
- Notes/PDF/image resources.

SECTION 32 — MATERIAL LIBRARY

Material cards should clearly show:

- file type;
- title;
- subject;
- date;
- size where relevant;
- saved state;
- sharing state where relevant.

Use static file illustrations/icons.

Actions through a menu may include:

Open
Ask AI
Share to Group
Rename
Download
Replace
Delete

Do not permanently display six buttons on every card.

SECTION 33 — DOCUMENT READER

Make the document/PDF reader simple.

Show:

- document title;
- page indicator;
- search if supported;
- bookmark/save;
- Ask AI;
- Share;
- Download/Offline if supported.

Keep most of the screen dedicated to reading.

SECTION 34 — AI ASSISTANT UX

AI must feel connected to study material.

Provide contextual actions such as:

- Ask about this PDF
- Summarize this material
- Explain this topic
- Explain this content simply
- Clean up these notes
- Extract key points
- Ask a general academic question

Distinguish:

General AI

from:

Material Context AI

When context exists show:

Using:
Database_Normalization.pdf

Allow the user to remove/change context.

SECTION 35 — AI VISUAL DESIGN

Use a static, professional AI illustration/icon.

Example:

- small sparkle + book;
- brain + document;
- study assistant mascot-like graphic if tasteful.

It must remain static.

Do not create:

- typing animation;
- bouncing dots;
- glowing AI orb;
- pulsing icon;
- animated bot.

For processing show clear text such as:

Reading your material...

or:

Preparing answer...

SECTION 36 — AI TECHNICAL VERIFICATION

Trace:

```
Flutter
↓
Firebase authentication
↓
AI endpoint
↓
FastAPI
↓
permission check
↓
material retrieval
↓
B2/file retrieval
↓
PDF/text/image extraction
↓
OCR if required
↓
Gemini
↓
FastAPI response
↓
Flutter rendering
```

Test:

1. General study question.
2. PDF summary.
3. PDF question.
4. Text-note question.
5. Image question.
6. Image-only PDF.
7. Large PDF.
8. Corrupt/invalid file.
9. Missing authentication.

- 10. Gemini failure.
- 11. Empty AI response.
- 12. AI quota reached.

=====

SECTION 37 — IMAGE-ONLY PDF

=====

This case is especially important.

If PDF extraction returns insufficient usable text:

```
PDF
↓
Detect little/no text
↓
Render pages to image
↓
OCR
↓
Extract text
↓
Prepare context
↓
Gemini
```

Use existing project architecture wherever possible.

Do not bypass authentication.

SECTION 38 — AI ERROR HANDLING

Never show an infinite loading screen.

Examples:

Could not read this document.

This appears to be a scanned PDF. OCR could not extract enough text.

AI usage limit reached for today.

The selected material is no longer available.

Unable to connect to AI service. Try again.

Do not expose backend exceptions.

SECTION 39 — TASK MANAGER

Verify and preserve:

- add;
- edit;
- complete;
- undo;
- delete;

- due date;
- reminder;
- priority if implemented.

Simple views may include:

Today
Upcoming
Completed

Do not introduce unnecessary Kanban complexity.

Use a static checklist illustration for appropriate empty states.

=====

SECTION 40 — STUDY PLANNER

=====

A planned study session may show:

- subject;
- task;
- date;
- time;
- duration;
- reminder;
- completion.

Use a clean agenda/timeline/calendar structure.

Avoid decorative complexity.

=====

SECTION 41 — FOCUS SESSION

Focus should be distraction-free.

Prioritize:

- selected subject;
- time;
- goal;
- start;
- pause/resume if existing;
- finish;
- history.

Do not add motion effects.

Use a static minimal focus/study illustration when no session is active.

SECTION 42 — STUDY GROUP STRUCTURE

Each group should be organized into:

Overview

Resources

Chat

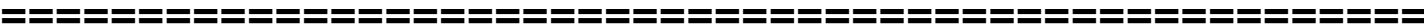
Members

where chat is enabled.

Do not put every group feature on one overloaded screen.



SECTION 43 — GROUP STATIC VISUALS



Use clean static illustrations/icons such as:

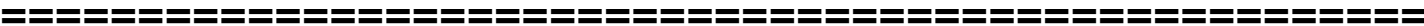
Group: students/books icon

Resources: shared folder

Chat: message bubbles

Members: people icon

No moving chat illustrations.



SECTION 44 — GROUP CHAT VERIFICATION



Verify:

- create group;
- join;
- leave;
- group membership;
- group authorization;
- send message;
- display message;
- timestamp;
- ordering;
- loading state;
- empty state;

- error recovery;
- mute behavior where implemented;
- unauthorized access prevention.

Do not create fake chat.

SECTION 45 — GROUP RESOURCE VERIFICATION

The user should be able to share every resource type supported by the backend.

Verify:

- PDF;
- image;
- text note;
- DOC/DOCX where supported;
- other allowed MIME types.

Do not allow Flutter to artificially block backend-supported formats.

Each resource should show:

- static file-type icon;
- name;
- uploader;
- date/time;
- size if useful;
- Open;
- Download/Save where supported.

Verify private access.

SECTION 46 — LIFE HUB

Life contains:



Do not overload Life with tiny unrelated modules.

SECTION 47 — EXPENSE ARCHITECTURE

There is ONE financial module:

EXPENSE

Do NOT expose:



and:



as unrelated top-level products.

Use:

```
Expense
|
├─ Overview
├─ Daily Expense
├─ Grocery / Bazar
├─ History / Calendar
├─ Budget
└─ Analytics
```

SECTION 48 — EXPENSE OVERVIEW

Immediately show:

```
Monthly Available
Total Spent
Remaining
```

Then:

- today's spending;
- recent transactions;
- grocery spending;
- simple category summary.

Use charts only if genuinely useful.

Avoid dashboard overload.

SECTION 49 — EXPENSE STATIC VISUALS

Use simple static illustrations/icons:

Expense: wallet/receipt

Budget: wallet/chart

Grocery: basket/bag

Calendar: calendar

Do not use animated money icons.

SECTION 50 — DAILY EXPENSE

Adding an expense should be extremely fast.

Preserve existing categories such as:

- Breakfast / Nasta
- Lunch
- Snacks
- Dinner
- Other

Do not change stored identifiers without migration.

SECTION 51 — GROCERY / BAZAR

Keep Bazar inside Expense.

Existing workflow:

```
Item
↓
Unit Price
↓
Quantity
↓
Automatic Total
↓
Purchased / Pending
```

Verify:

Purchased item → intended financial transaction exactly once.

Unpurchase → corresponding transaction removed/reversed according to existing logic.

Delete → no orphan financial transaction.

Retry → no duplicate financial transaction.

Do not reintroduce Bazar OCR.

SECTION 52 — MEDICINE HOME

Prioritize:

Today's Medicines
Next Reminder
Taken
Skipped
History
Add Medicine
Scan Prescription

Manual entry must always remain available.

SECTION 53 — MEDICINE STATIC VISUALS

Use clean static illustrated icons:

Medicine: pill/medicine box

Prescription: paper + medicine symbol

Reminder: clock + medicine

Taken: checked medicine icon

Skipped: simple skipped state

Avoid overly realistic medical imagery.

SECTION 54 — PRESCRIPTION OCR TECHNICAL AUDIT

Trace:

```
Flutter image/PDF
↓
Authenticated upload
↓
FastAPI
↓
Image preprocessing
↓
OCR
↓
Text parsing
↓
Possible medicine results
↓
Flutter review
↓
Student correction
↓
Student confirmation
↓
Medicine save
```

Determine exactly:

1. Which OCR engine is used?
2. Does preprocessing exist?
3. Is English supported?
4. Is Bangla supported?
5. Which language data is installed?
6. Is there a prescription parser?
7. Is it regex/rule based?
8. Is Gemini involved?
9. Is a custom ML model present?

10. Is that model executed?
11. Where do confidence percentages come from?
12. Are confidence values scientifically generated or manually/fallback generated?

Do not fabricate confidence.

SECTION 55 — OCR VS ML TERMINOLOGY

The final documentation must distinguish:

A. OCR Engine

Example:

Tesseract

B. Prescription Parser

Example:

Regex / dictionary / entity matching

C. AI Structured Extraction

Example:

Gemini processing OCR text

D. Custom ML Model

Example:

A separately trained prescription medicine extraction model

Do not describe A or B as a custom ML model unless technically accurate.

SECTION 56 — IF PRESCRIPTION ML EXISTS

If a real model exists:

Trace:

```
Model artifact
↓
Loader
↓
Preprocessor
↓
Inference
↓
Postprocessor
↓
Prescription service
↓
API route
↓
Flutter
```

If it exists but is disconnected:

connect it through the existing architecture.

Do not create a duplicate endpoint unless required.

=====

SECTION 57 — IF NO CUSTOM PRESCRIPTION ML EXISTS

=====

Do not pretend one exists.

Use the existing OCR architecture honestly.

If Gemini is already available server-side and can safely assist with structuring extracted text, this may be integrated internally when compatible with the current backend.

But:

OCR/AI output remains a suggestion.

Never automatically create:

- medicine;
- dose;
- schedule;
- reminder;
- price.

Student confirmation is mandatory.

Do NOT build a meaningless ML model using fake training data simply so the project can claim "ML."

=====

SECTION 58 — MEDICINE SAFETY

=====

Display:

OCR may be inaccurate. Verify medicine information before saving.

Also:

Gochano does not provide medical advice.

Never hide uncertainty.

SECTION 59 — MEDICINE REMINDERS

Verify:

- reminder creation;
- scheduled time;
- notification;
- Taken;
- Skipped;
- medicine history.

Keep reminder screens simple.

SECTION 60 — COMMUTE BD PRIMARY UX

CommuteBD should feel like a modern Bangladesh transport assistant.

Primary flow:

From
↓
To
↓
Find Routes

Then present:

Recommended
Cheapest
Fastest

Route summary:

- duration;
- distance;
- fare;
- transport modes;
- transfer count;
- source/estimate type where useful.

SECTION 61 — COMMUTE STATIC VISUALS

Use static illustrated icons for:

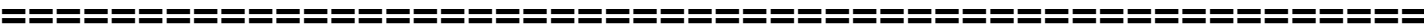
- walking;
- bus;
- metro;
- rickshaw;
- auto;
- CNG;
- train;
- boat/launch;
- destination;
- map pin.

Icons should be easy to distinguish at a glance.

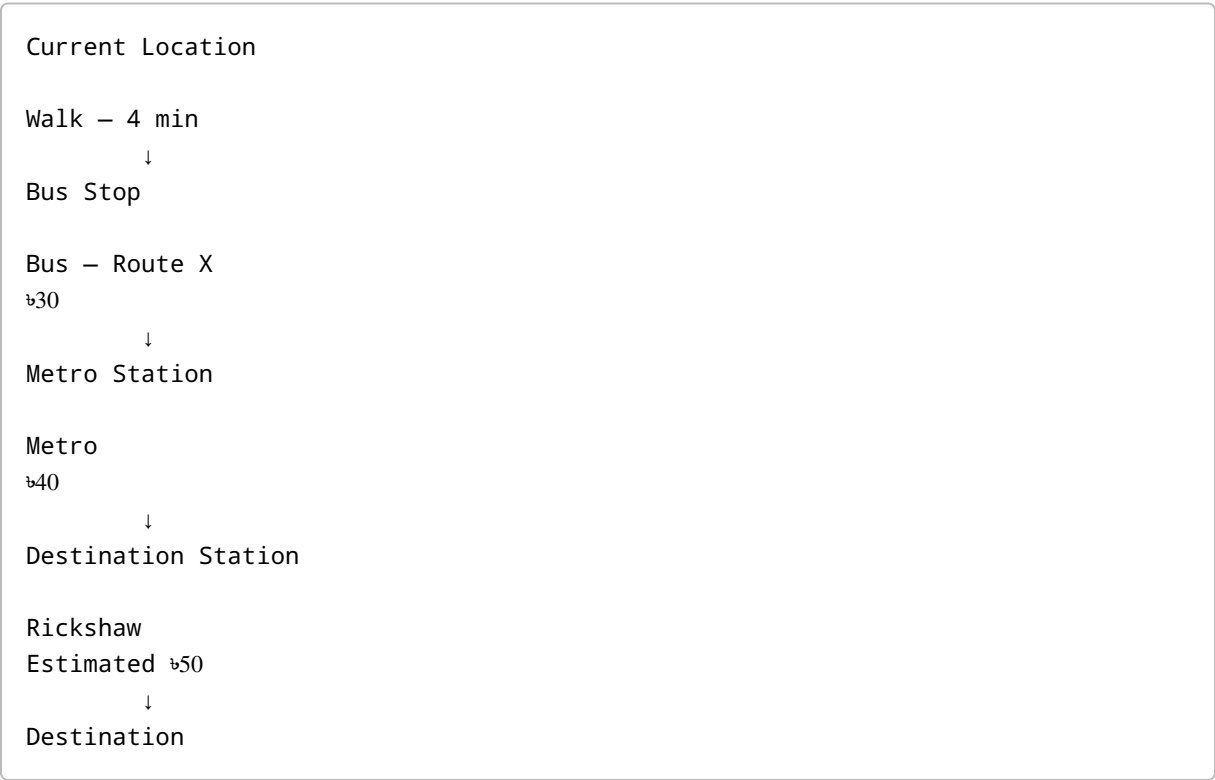
Use consistent illustrated/cartoon-inspired transport graphics without making them childish.



SECTION 62 — MULTIMODAL ROUTE PRESENTATION

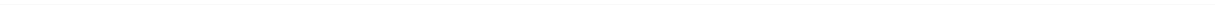


Example:



Use a static connecting line / route hierarchy.

Do NOT animate the route line.



SECTION 63 — DIJKSTRA VERIFICATION

The project expects shortest-path logic involving Dijkstra.

DO NOT simply state that it uses Dijkstra.

Search actual code.

Find:

- graph construction;
- nodes;
- edges;
- edge weights;
- shortest-path function;
- Dijkstra implementation;
- NetworkX usage if any;
- alternatives;
- PostGIS queries;
- OSRM interaction.

Trace:

```
Route Request
↓
Graph Selection
↓
Algorithm
↓
Route Result
↓
API
↓
Flutter
```

Confirm whether Dijkstra actually executes.

SECTION 64 — IF DIJKSTRA IS DISCONNECTED

If Dijkstra exists but the production route flow bypasses it:

repair the internal connection.

Preserve:

- API endpoint;
- authentication;
- schemas;
- existing databases.

If another routing algorithm is intentionally used in specific circumstances, document it accurately.

SECTION 65 — COMMUTE ML FARE MODEL AUDIT

Search for:

- training notebook/script;
- training CSV;
- model artifact;
- features;
- loader;
- predict;
- route integration;
- fallback mechanism.

Trace:

Transport Mode
+

```
Distance
+
Route features
+
Other input features
    ↓
Fare predictor
    ↓
Prediction
    ↓
API response
    ↓
Flutter
```

Determine whether prediction actually happens at runtime.

SECTION 66 — IF COMMUTE ML MODEL EXISTS

If real fare ML exists but is disconnected:

connect it.

Do not redesign the API.

Provide a reliable fallback if inference fails.

Do not return:

0

or fabricated values just to avoid errors.

SECTION 67 — IF COMMUTE ML DOES NOT EXIST

Do not claim it exists.

Determine whether the repository contains enough:

- training data;
- defined features;
- training code;
- model implementation;

to complete the originally intended model.

If implementation is substantially present but unfinished, complete the connection internally.

If no valid model/data exist:

retain deterministic/statistical fare estimation.

Document honestly:

Custom fare ML model not currently available.

Do not create meaningless ML from insufficient data.

SECTION 68 — FARE RELIABILITY

Differentiate:

Official / Deterministic

For data such as:

- Metro;
- official fares;
- known transport tables.

Estimated

For modes such as:

- Rickshaw;
- Auto;
- CNG;
- variable routes.

Estimated fares may use:

- distance;
- historical reports;
- crowd data;
- statistical rules;
- verified ML prediction where actually implemented.

Never label estimated values as official.

=====

SECTION 69 — POST-TRIP FARE CONFIRMATION

=====

Estimated fare must not automatically enter Expense.

Flow:

```
Route Estimate
↓
Trip
↓
Actual Fare
```

↓
Student Confirmation
↓
Fare Report
↓
Expense integration where existing logic permits

Check deduplication.

SECTION 70 — COMMUNITY

Community should remain student-focused.

Prioritize:

- academic questions;
- student discussions;
- campus information;
- helpful posts;
- announcements;
- comments.

Do not redesign it as:

- Facebook;
- TikTok;
- Instagram;
- influencer platform.

Avoid unnecessary:

- follower count prominence;
- reels;
- engagement tricks.

SECTION 71 — COMMUNITY STATIC VISUALS

Use small static illustrations for:

- discussions;
- questions;
- announcements;
- empty states.

Keep content itself visually dominant.

SECTION 72 — PROFILE

Organize Profile into clear sections:

Account

Monthly Money / Financial Settings

Statistics

Language

Appearance

Notifications

Storage / Usage where available

Privacy

Data Export/Delete where available

About

Do not expose developer configuration to normal users.

SECTION 73 — BILINGUAL EXPERIENCE

Verify:

English
Bangla

Only the selected language should display at once.

Audit:

- bottom navigation;
- page titles;
- forms;
- buttons;
- dialogs;
- validation;
- errors;
- empty states;
- Study;
- Groups;
- AI;
- Expense;
- Medicine;
- CommuteBD;
- Community;
- Profile;
- notifications.

Avoid mixed-language accidental strings.

Avoid clipped Bangla text.

SECTION 74 — LOADING STATES

Every network-driven screen needs clear processing feedback.

Examples:

Loading subjects...

Uploading 45%

Reading document...

Checking route...

Scanning prescription...

Avoid unnecessary decorative loaders.

Never leave the screen looking frozen.

SECTION 75 — EMPTY STATES

Every important list should have a deliberate empty state.

Examples:

No subjects yet
Create your first subject to organize your study materials.

No materials yet
Add your first note, PDF or study resource.

No tasks today
Your schedule is clear.

No shared resources
Share the first resource with your group.

No expenses today
Nothing has been recorded yet.

Use a small appropriate static illustration.

SECTION 76 — ERROR STATES

Never show raw technical errors to normal students.

Do not show:

- stack trace;
- SQL error;
- Firebase exception;
- DioException;
- Python traceback.

Use:

Something went wrong.
Try again.

or more specific understandable messages.

Keep technical information in logs.

SECTION 77 — OFFLINE / BAD NETWORK

Where existing architecture supports it:

- retain previously loaded content;
- indicate connectivity issue;
- allow retry;
- avoid duplicate submissions;
- avoid clearing forms unnecessarily.

Check especially:

- Tasks;
 - Expense;
 - Materials;
 - Groups;
 - AI;
 - Medicine;
 - Commute.
-

SECTION 78 — NOTIFICATIONS

Primary intended notifications:

- Medicine reminders;
- Study/task reminders;
- Upcoming study sessions;
- Private Study Group notifications where enabled.

Optional:

- budget threshold alert.

Avoid noisy notifications for:

- ordinary expense entry;
- normal grocery change;
- normal commute calculation.

SECTION 79 — BACKBLAZE B2 AUDIT

Trace:

```
Flutter
↓
Authenticated FastAPI
↓
Ownership/quota validation
↓
Backblaze B2
↓
Private object
```

```
↓  
Metadata  
↓  
Signed/authorized access  
↓  
Flutter
```

Search for old competing Firebase Storage implementations.

If both exist:

determine which functions are actually imported and executed.

Final intended private file provider:

Backblaze B2

Do not leave runtime storage ambiguous.

SECTION 80 — FIREBASE AUDIT

Verify:

- correct Firebase project;
- Firebase initialization;
- Auth;
- Firestore;
- Firebase Admin;
- FCM;
- token verification.

Make sure Flutter and backend refer to the intended same production Firebase project.

Do not silently mix multiple Firebase project configurations.

SECTION 81 — NEON / POSTGIS AUDIT

Verify:

- DATABASE_URL;
- connection;
- schema;
- PostGIS extension;
- migration state;
- transport tables;
- spatial columns;
- indexes;
- CommuteBD queries.

Do not modify production data destructively.

SECTION 82 — SECURITY

Never place in Flutter:

```
GEMINI_API_KEY
DATABASE_URL
B2_APPLICATION_KEY
B2_KEY_ID
Firebase Admin Private Key
FIREBASE_SERVICE_ACCOUNT_B64
Android signing password
```

Protected FastAPI routes must use Firebase authentication.

Verify user ownership before returning private files.

Verify group membership before returning group resources.

SECTION 83 — PERFORMANCE

The redesigned application must remain fast.

Avoid:

- huge images;
- unnecessary illustrations;
- repeated Firestore reads;
- repeated API calls;
- excessive rebuilds;
- loading complete Bangladesh datasets in Flutter;
- unnecessary dependency packages;
- giant lists without lazy loading;
- oversized raster assets.

Optimize static illustrations.

Prefer:

- SVG/vector assets where appropriate;
 - WebP where raster is necessary;
 - thumbnails;
 - cached network images where appropriate;
 - const widgets;
 - lazy `ListView.builder`;
 - proper state boundaries.
-

SECTION 84 — CENTRAL UI COMPONENT LIBRARY

Create/reuse consistent components such as:

```
GochanoScaffold
GochanoAppBar
GochanoBottomNavigation
SectionHeader
PrimaryButton
SecondaryButton
IconActionButton
AppCard
StatCard
SubjectIllustration
ResourceIcon
MaterialCard
TaskTile
GroupCard
GroupResourceCard
ExpenseSummaryCard
MedicineCard
CommuteRouteCard
```

AIContextChip
SearchField
FilterChip
EmptyState
ErrorState
StaticLoadingState
ConfirmationSheet

Adapt names to existing conventions if necessary.

Do not create duplicate variants of nearly identical components.



SECTION 85 — NAVIGATION QUALITY



For every screen answer:

- Where am I?
- How do I return?
- What is the main action?
- What is secondary?
- Where can I find related features?

Avoid unnecessarily deep navigation.

Preserve state between main destinations where practical.

SECTION 86 — MINIMALISM RULE

For every screen ask:

Can this screen be simpler without losing functionality?

Remove:

- duplicate headings;
- duplicate explanations;
- unnecessary cards;
- unnecessary borders;
- repeated information;
- excessive icons;
- unnecessary CTA buttons;
- decorative clutter.

SECTION 87 — MINIMALISM DOES NOT MEAN REMOVING FEATURES

Do NOT delete features to achieve simplicity.

Use:

- overflow menus;
- tabs;
- bottom sheets;
- context menus;
- filters;
- progressive disclosure;

- expandable information where suitable.

Keep full functionality.

SECTION 88 — FEATURE VERIFICATION MATRIX

Verify all of the following.

AUTHENTICATION

- ☐ Register
 - ☐ Login
 - ☐ Logout
 - ☐ Session restore
 - ☐ Invalid token handling
 - ☐ Profile loading
-

STUDY

- ☐ Create semester
- ☐ Edit semester where supported
- ☐ Delete semester where supported
- ☐ Create subject
- ☐ Edit subject where supported
- ☐ Open subject
- ☐ Add material

- ☐ PDF upload
 - ☐ Image upload
 - ☐ Note
 - ☐ Other backend-supported file types
 - ☐ Open material
 - ☐ Search
 - ☐ Rename
 - ☐ Delete
 - ☐ Download
 - ☐ Bookmark/save
 - ☐ Recent materials
 - ☐ Share to group
-

AI

- ☐ General question
- ☐ PDF context
- ☐ Image context
- ☐ Note context
- ☐ Scanned/image-only PDF
- ☐ Summary
- ☐ Explanation
- ☐ Invalid material
- ☐ Authentication

☐ AI quota

☐ Provider failure

TASKS

☐ Add

☐ Edit

☐ Complete

☐ Undo

☐ Delete

☐ Reminder

STUDY PLANNER

☐ Create session

☐ Edit

☐ Complete

☐ Reminder

FOCUS

☐ Start

☐ Pause/resume if supported

☐ Finish

☐ History

STUDY GROUPS

- ☐ Create
 - ☐ Join
 - ☐ Leave
 - ☐ Member list
 - ☐ Authorization
 - ☐ Chat
 - ☐ PDF sharing
 - ☐ Image sharing
 - ☐ Text/note sharing
 - ☐ DOC/DOCX where backend-supported
 - ☐ Open resource
 - ☐ Download/save
 - ☐ Unauthorized resource prevention
-

EXPENSE

- ☐ Monthly money
- ☐ Add daily expense
- ☐ Edit/delete where supported
- ☐ Daily total
- ☐ Monthly total
- ☐ Remaining amount
- ☐ Calendar/history

- ☐ Grocery item
 - ☐ Unit price
 - ☐ Quantity
 - ☐ Automatic total
 - ☐ Purchased status
 - ☐ Financial transaction mirror
 - ☐ Unpurchase
 - ☐ Delete
 - ☐ Duplicate prevention
-

MEDICINE

- ☐ Manual add
 - ☐ Prescription image
 - ☐ Prescription PDF where supported
 - ☐ OCR
 - ☐ OCR preprocessing
 - ☐ Medicine extraction
 - ☐ Manual correction
 - ☐ Confirmation
 - ☐ Reminder
 - ☐ Taken
 - ☐ Skipped
 - ☐ History
-

COMMUTEBD

- ☐ Origin search
 - ☐ Destination search
 - ☐ Location/geocoding
 - ☐ Recommended route
 - ☐ Cheapest route
 - ☐ Fastest route
 - ☐ Multimodal path
 - ☐ Dijkstra verification
 - ☐ OSRM integration
 - ☐ Fare model
 - ☐ Fare fallback
 - ☐ Route geometry
 - ☐ Post-trip actual fare
 - ☐ Fare report
 - ☐ Expense connection
 - ☐ Duplicate prevention
-

COMMUNITY

- ☐ Load posts
- ☐ Create post
- ☐ Comment
- ☐ Permissions

[] Empty state

[] Error state

PROFILE

[] Account

[] Language

[] Theme

[] Notifications

[] Monthly money/settings

[] Statistics

[] Privacy/data actions where implemented

SECTION 89 — OLD UI VS NEW UI PARITY

Before replacing any old screen:

Create an internal checklist:

OLD SCREEN FUNCTIONS

vs.

NEW SCREEN FUNCTIONS

Every working action must survive unless explicitly obsolete.

Only remove the old UI after parity is verified.

=====

SECTION 90 — DO NOT HIDE BROKEN FEATURES

=====

If a feature is broken:

investigate it.

Do not:

- hide the button;
- delete the page;
- display "Coming Soon";
- return fake output;
- return hard-coded fare;
- return hard-coded AI answers;
- claim OCR success without OCR;
- claim ML prediction without inference.

Fix the wiring where possible.

=====

SECTION 91 — FLUTTER QUALITY CHECK

=====

After UI restructuring run:

```
flutter pub get
```

Then:

```
flutter analyze
```

Then:

```
flutter test
```

Resolve relevant failures.

If the Android environment permits:

```
flutter build apk --debug
```

or the repository's appropriate Android build command.

SECTION 92 — BACKEND QUALITY CHECK

If backend code is modified:

Run the repository's backend tests.

For example:

```
pytest
```

Do not claim tests passed without actually executing them.

SECTION 93 — REAL INTEGRATION CHECK

Where environment credentials are available, verify:

- Firebase Auth;
- Firestore;
- Firebase Admin;
- FCM;
- B2;
- Gemini;
- Neon;
- PostGIS;
- OCR;
- PDF processing;
- Commute route;
- fare prediction;
- Group resources;
- Group chat.

Do not fake success if a production credential is unavailable.

Report:

Configuration required

where appropriate.

SECTION 94 — VISUAL QA

Inspect major screens in:

- Light Mode

- Dark Mode
- English
- Bangla
- Small-screen Android
- Normal Android

Check:

- overflow;
- text clipping;
- alignment;
- illustration scaling;
- spacing;
- keyboard;
- dialogs;
- sheets;
- navigation;
- long file names;
- long subject names;
- Bangla strings;
- errors;
- empty states.

SECTION 95 — STATIC ASSET QUALITY

Audit every static illustration/icon.

Make sure:

- consistent style;
- consistent stroke/shape language;
- appropriate resolution;
- no pixelation;
- no random unrelated icon packs;
- light/dark compatibility;
- no huge APK increase;
- no copyrighted asset misuse.

Prefer internally consistent open/licensed or project-owned visual assets.

Document asset sources/licenses where required.

SECTION 96 — REMOVE DEAD FRONTEND CODE

After confirming no references:

remove:

- obsolete old screens;
- abandoned prototype pages;
- duplicate widgets;
- duplicate theme code;
- obsolete navigation destinations;
- unused static assets;
- dead UI logic.

Do NOT delete active backend/service code merely because an old page is gone.

SECTION 97 — README CLEANUP

At the END:

Search entire repository for:

```
README
README.md
README_*.md
readme.md
ReadMe.md
```

and duplicate project explanation files that function as alternate READMEs.

There should ultimately be:

ONE AUTHORITATIVE ROOT README.md

Before deleting duplicate README files:

extract still-valid information.

Merge useful content into the final README.

Then remove outdated duplicates.

Do not delete:

- LICENSE;
- legally required attribution;
- database migrations;
- necessary runbook files if they are operational rather than duplicate README material.

However, the root README should be the single primary project explanation.

SECTION 98 — FINAL README = FULL PROJECT REPORT

README.md must act as:

```
Project Report
+
Architecture Documentation
+
Developer Guide
+
Project Owner Guide
+
```

Setup Guide
+
Feature Guide
+
Testing Guide
+
Deployment Guide
+
Troubleshooting Guide
+
Verified Status Report

Do NOT make it a tiny GitHub README.

SECTION 99 — FINAL README STRUCTURE

Use approximately this structure.

Gochano

1. Project Overview

Explain Gochano.

2. Problem Statement

3. Project Objectives

4. Target User

Student-focused.

5. Main Application Structure

Home
Study

Life
Community
Profile

6. Complete Feature Overview

7. Home

8. Study Workspace

9. Notes / PDF / Materials

10. Tasks

11. Planner

12. Focus Sessions

13. AI Assistant

14. Study Groups

15. Group Chat

16. Group Resource Sharing

17. Expense

18. Grocery / Bazar

19. Medicine

20. Prescription OCR

21. CommuteBD

22. Community

23. Profile

24. UI/UX Design System

Explain:

- Clean Minimalism;

- static illustrated icons;
- static cartoon-inspired subject visuals;
- responsive system;
- light/dark mode;
- accessibility.

Explicitly state that decorative animations are intentionally not part of the UI design.

25. Complete Technology Stack

26. System Architecture

Provide architecture diagram.

27. Data Ownership

Explain which platform owns which information.

28. Authentication Flow

29. Firestore Architecture

30. Backblaze B2 Architecture

31. AI Architecture

32. PDF/Material AI Flow

33. OCR Architecture

Be technically accurate.

State whether it uses:

- Tesseract;
- preprocessing;
- rules;
- Gemini;
- custom ML.

34. ML Model Inventory

Mandatory.

For every detected ML model include:

Name:
Purpose:
Model file:
Framework:
Input:
Output:
Loader:
Backend service:
Endpoint:
Flutter feature:
Runtime status:

Especially:

- Commute fare model;
- Prescription ML model if any.

35. CommuteBD Architecture

Explain:

- datasets;
- graph;
- nodes;
- edges;
- Dijkstra status;
- OSRM;
- PostGIS;
- fare system;
- ML prediction;
- fallbacks;
- crowd reports.

36. Expense Architecture

Explain ledger synchronization.

37. Medicine Architecture

38. Notification Architecture

39. Localization

40. Folder Structure

41. Environment Variables

List names only.

Never secret values.

Potential variables include:

```
APP_ENV  
  
FIREBASE_PROJECT_ID  
  
FIREBASE_SERVICE_ACCOUNT_B64  
  
DATABASE_URL  
  
GEMINI_API_KEY  
  
GEMINI_MODEL  
  
B2_BUCKET_NAME  
  
B2_ENDPOINT_URL  
  
B2_REGION  
  
B2_KEY_ID  
  
B2_APPLICATION_KEY  
  
MAX_UPLOAD_MB  
  
USER_STORAGE_LIMIT_MB  
  
UPLOAD_DAILY_LIMIT  
  
AI_DAILY_LIMIT
```

SIGNED_URL_TTL_SECONDS

Only document variables actually relevant to current code.

Remove obsolete variable documentation.

42. Local Development Setup

43. Firebase Setup

44. Neon/PostGIS Setup

45. Backblaze B2 Setup

46. Gemini Setup

47. Render Setup

48. Android Setup

49. How to Run Flutter

50. How to Run Backend

51. Testing

Only state VERIFIED current test results.

Do not reuse old test numbers without running tests.

52. Feature Verification Table

Columns:

Feature
Status
Implementation
Dependency
Limitations

Statuses:

Verified

Working with limitation

Configuration required

Not available

Needs production validation

53. Security

54. Privacy

55. Known Limitations

56. Troubleshooting

57. What the Project Owner Must Do

MANDATORY.

Clearly explain what remains for me personally.

Example:

1. Configure production environment variables
2. Verify Firebase project
3. Verify B2 bucket
4. Verify Neon production database
5. Add Gemini API key
6. Test notifications
7. Perform real-device test
8. Generate signed release APK

Only include tasks actually required.

58. What the Application Does Automatically

Separate this from owner responsibilities.

59. How Every Feature Works

User instructions:

How to create semester

How to add subject

How to upload PDF

How to create note

How to ask AI about material

How to create group

How to chat

How to share PDF/resource

How to create task

How to create study session

How to add expense

How to use Grocery/Bazar

How to add medicine manually

How to scan prescription

How to verify OCR

How to mark Taken/Skipped

How to use CommuteBD

How to submit actual fare

How to use Community

60. Deployment Checklist

61. Production Readiness Checklist

62. Future Scope

63. Final Project Status

Describe status honestly.

=====

SECTION 100 — VERIFIED CLAIMS ONLY

=====

README must NOT say:

AI works perfectly

unless tested.

Do NOT say:

Uses Dijkstra

unless runtime path is verified.

Do NOT say:

Machine-learning fare prediction

unless model inference is verified.

Do NOT say:

ML-powered prescription detection

unless a custom ML model actually exists.

Do NOT say:

Production Ready

unless production integrations have been validated.

Use precise wording.

Examples:

Verified locally.

Requires production configuration.

Dijkstra implementation exists and is executed by the route service.

No custom prescription ML model detected. Current implementation uses Tesseract OCR and rule-based parsing.

Fare model artifact exists but production inference has not been validated.

SECTION 101 — PROJECT OWNER VS CODE

README must clearly separate:

CODE RESPONSIBILITIES

from:

PROJECT OWNER RESPONSIBILITIES

For each external platform tell me exactly what I need to configure manually.

Do not tell me to change source code when an environment variable or dashboard setting is enough.

SECTION 102 — REDESIGN ACCEPTANCE CRITERIA

The UI redesign is complete only if:

- ☐ Entire application follows one Clean Minimalist design system.
- ☐ Decorative animation has NOT been introduced.
- ☐ Subject visuals are static illustrated/cartoon-inspired icons.
- ☐ Static illustrations are consistent across the app.
- ☐ Home is simplified.
- ☐ Bottom navigation contains five main destinations.
- ☐ Study hierarchy is clear.
- ☐ Material library is clean.
- ☐ PDF reader is uncluttered.
- ☐ AI context is understandable.
- ☐ Tasks are simple.

- ☐ Planner is understandable.
- ☐ Focus interface is distraction-free.
- ☐ Groups clearly separate Chat, Resources and Members.
- ☐ Group chat works.
- ☐ Group resources work.
- ☐ PDF sharing works.
- ☐ Expense is unified.
- ☐ Grocery exists inside Expense.
- ☐ Medicine uses safety-first OCR.
- ☐ Commute is understandable.
- ☐ Community is simple.
- ☐ Profile/settings are organized.
- ☐ Light mode works.
- ☐ Dark mode works.
- ☐ English works.
- ☐ Bangla works.
- ☐ Static illustrations scale correctly.
- ☐ Loading states exist.
- ☐ Empty states exist.
- ☐ Error states exist.
- ☐ No major overflow.
- ☐ No broken navigation.
- ☐ No broken buttons.

[] No fake screens.

[] No functionality removed for visual simplicity.

SECTION 103 — FUNCTIONAL ACCEPTANCE CRITERIA

Before completion:

AI

Must be traced and tested.

Groups

Membership, chat and resources must be traced and tested.

Resource Sharing

Backend-supported file types must work from Flutter.

Commute

Routing must be traced.

Dijkstra

Actual usage must be confirmed.

Fare ML

Model existence and runtime inference must be confirmed.

OCR

Engine, preprocessing, parser and ML status must be confirmed.

B2

Actual runtime storage provider must be confirmed.

Expense

Financial ledger duplication must be checked.

Medicine

OCR confirmation and reminders must be checked.

Firebase

Configuration consistency must be checked.

Neon/PostGIS

Connection/schema readiness must be checked.

SECTION 104 — IMPLEMENTATION ORDER

Follow approximately this order.

PHASE A — Repository Audit

Inspect entire project.

PHASE B — Existing Feature Map

Map screens → services → APIs → backend.

PHASE C — Integration Audit

Identify broken/disconnected features.

PHASE D — Static Visual System

Create:

- design tokens;
- typography;
- colors;
- static illustration/icon system;
- subject illustration mapping;
- empty-state illustrations.

PHASE E — Navigation Shell

Rebuild primary application shell.

PHASE F — Home

Redesign dashboard.

PHASE G — Study

Rebuild:

- semesters;
- subjects;
- materials;
- document reading.

PHASE H — AI

Redesign AI UI and verify all AI paths.

PHASE I — Tasks / Planner / Focus

PHASE J — Study Groups

Verify:

- groups;
- chat;

- resources;
- sharing;
- authorization.

PHASE K — Expense

Unify daily expense and Grocery/Bazar.

PHASE L — Medicine

Redesign and audit OCR.

PHASE M — CommuteBD

Redesign Commute UI and audit:

- routing;
- Dijkstra;
- fare calculation;
- ML prediction;
- PostGIS;
- OSRM.

PHASE N — Community

PHASE O — Profile

PHASE P — Localization

PHASE Q — Dark Mode

PHASE R — Accessibility

PHASE S — Performance

PHASE T — Complete Testing

PHASE U — Remove Dead UI

PHASE V — README Cleanup

PHASE W — Generate Final README.md

PHASE X — Final Verification Report

=====

SECTION 105 — THIS IS NOT A MOCKUP TASK

=====

Do not only make attractive screens.

The redesigned UI must use real functionality.

Buttons must call actual services.

Forms must save real data.

Uploads must reach actual storage.

AI must use the real AI backend.

Chat must use real group data.

Resources must open real resources.

Expense must persist.

Medicine must persist.

CommuteBD must use the actual route backend.

The deliverable is:

WORKING REDESIGNED APPLICATION

not:

STATIC UI DEMO.

=====

SECTION 106 — FINAL VERIFICATION REPORT

=====

At completion provide a concise report including:

1. UI architecture changes.
2. Main screens redesigned.
3. Design-system components created.
4. Static illustrations/icons introduced.
5. Subject visual mapping created.
6. Features verified.
7. Bugs found.

8. Bugs fixed.
9. AI status.
10. PDF AI status.
11. Image-only PDF status.
12. Group status.
13. Group-chat status.
14. Group-resource status.
15. PDF sharing status.
16. OCR engine status.
17. OCR preprocessing status.
18. Prescription parser status.
19. Prescription custom-ML status.
20. Commute route status.
21. Dijkstra status.
22. Commute fare ML status.
23. OSRM status.
24. Neon/PostGIS status.
25. Backblaze B2 status.
26. Firebase status.
27. FCM/reminders status.
28. Expense ledger status.
29. Flutter analyze result.
30. Flutter test result.

- 31. Backend test result if backend changed.
- 32. Android build result.
- 33. Remaining production limitations.
- 34. Actions still required from the project owner.

SECTION 107 — TARGET PRODUCT EXPERIENCE

The final Gochano application should feel like:

A calm personal operating system for student life.

Not:

A random collection of tools.

A student opening Gochano should quickly understand:

What should I study today?

Where are my materials?

What tasks are pending?

What are my study groups doing?

How can I ask AI about this material?

How much money do I have left?

What medicine is next?

How should I reach my destination?

SECTION 108 — FINAL VISUAL PRINCIPLES

Every visual decision should follow:

CLARITY
>
DECORATION

USABILITY
>
VISUAL NOVELTY

CONSISTENCY
>
SCREEN-BY-SCREEN EXPERIMENTATION

STATIC MEANINGFUL ILLUSTRATION
>
ANIMATION

PROFESSIONAL STUDENT EXPERIENCE
>
CHILDISH CARTOON DESIGN

REAL FUNCTIONALITY
>
DEMO APPEARANCE

STUDENT WORKFLOW
>
FEATURE COUNT

SECTION 109 — FINAL STATIC ILLUSTRATION

PRINCIPLE

Static illustrations should be used only where they improve:

- recognition;
- navigation;
- subject identity;
- empty states;
- feature discoverability;
- friendliness.

Use them for things such as:

Subject icons
Notes
PDFs
AI
Groups
Tasks
Expense
Grocery
Medicine
Prescription
Commute
Community
Empty states

Keep them:

- static;
- lightweight;
- consistent;
- clean;
- modern;
- slightly cartoon-inspired;
- professional.

Do NOT turn the complete application into an illustration-heavy children's interface.

=====

SECTION 110 — FINAL INSTRUCTION

=====

Start by inspecting the repository.

DO NOT immediately start changing screens.

First understand:

- what currently exists;
- what currently works;
- what is connected;
- what is broken;
- what features each existing screen contains.

Do not ask me to explain functionality that can be discovered from the repository.

Do not trust old README documents more than runtime code.

Working code and actual integration paths are stronger evidence than outdated documentation.

Then:

1. Rebuild the frontend information architecture.
2. Implement one Clean Minimalist design system.
3. Use static illustrated/cartoon-inspired visual icons where useful.
4. DO NOT introduce decorative animations.
5. Rebuild Home.
6. Rebuild Study.
7. Rebuild Subjects and Materials.

8. Rebuild AI experience.
9. Verify Gemini completely.
10. Rebuild Tasks/Planner/Focus.
11. Rebuild Groups.
12. Verify group chat.
13. Verify resource sharing.
14. Verify every backend-supported resource format.
15. Rebuild Expense.
16. Preserve financial ledger integrity.
17. Rebuild Medicine.
18. Audit OCR.
19. Verify whether prescription custom ML actually exists.
20. Connect it if it exists but is disconnected.
21. Do not fabricate ML if it does not exist.
22. Rebuild CommuteBD.
23. Verify Dijkstra.
24. Verify shortest-path runtime behavior.
25. Verify fare ML model.
26. Connect existing fare model if disconnected.
27. Keep honest fallbacks when no ML model exists.
28. Verify Neon/PostGIS.
29. Verify Backblaze B2.
30. Verify Firebase consistency.

31. Rebuild Community.
32. Rebuild Profile.
33. Verify Bangla/English.
34. Verify dark/light mode.
35. Verify responsiveness.
36. Run Flutter analysis/tests.
37. Run backend tests if backend internals were modified.
38. Build Android application where environment permits.
39. Remove obsolete frontend UI.
40. Remove duplicate/outdated README files.
41. Leave exactly ONE authoritative root README.md.

The final README must explain:

WHAT GOCHANO IS

WHY IT EXISTS

HOW THE APP IS ORGANIZED

HOW EVERY FEATURE WORKS

HOW THE FRONTEND IS STRUCTURED

HOW THE BACKEND IS STRUCTURED

HOW AUTHENTICATION WORKS

HOW FIRESTORE WORKS

HOW BACKBLAZE B2 WORKS

HOW AI WORKS

HOW PDF AI WORKS

HOW GROUPS WORK

HOW CHAT WORKS

HOW RESOURCE SHARING WORKS

HOW EXPENSE WORKS

HOW MEDICINE WORKS

HOW OCR WORKS

WHETHER OCR USES CUSTOM ML

HOW COMMUTEBD WORKS

WHETHER DIJKSTRA IS ACTUALLY USED

HOW FARE CALCULATION WORKS

WHETHER FARE PREDICTION USES REAL ML

WHICH ML MODELS ACTUALLY EXIST

HOW NEON/POSTGIS WORKS

HOW TO CONFIGURE FIREBASE

HOW TO CONFIGURE B2

HOW TO CONFIGURE NEON

HOW TO CONFIGURE GEMINI

HOW TO CONFIGURE RENDER

HOW TO RUN THE FRONTEND

HOW TO RUN THE BACKEND

HOW TO TEST EVERYTHING

HOW TO BUILD THE APK

WHAT I PERSONALLY STILL NEED TO DO

KNOWN LIMITATIONS

PRODUCTION READINESS

FINAL VERIFIED PROJECT STATUS

The final result must be a significantly more professional, structured, clean, minimal, intuitive and student-friendly Gochano application.

Its personality should come from:

- Clean layout
- +
- Strong typography
- +
- Excellent usability
- +
- Static subject illustrations
- +
- Static feature illustrations
- +
- Consistent icons
- +
- Professional color system

NOT from animation.

The redesigned Gochano should look like a polished real application while keeping the existing backend, databases, authentication, AI, storage, ML/routing architecture and business logic intact.